Threat Hunt Report: Operation QUARTZFANG

Organisation: SemiCrest Technologies (training scenario)
Platform: Microsoft Sentinel, workspace LAW-HuntPractice
Hunt window: 20 September 2026, 06:30 - 08:30 UTC
Hosts in scope: sc-portal01 (10.5.0.10), sc-db01 (10.5.0.20), sc-registry01 (10.5.0.30), sc-admin01 (10.5.0.40)
Scenario basis: Campaign C0062 (Anthropic threat intelligence, November 2025)
Report date: November 2026

---

1. Summary

I've been studying cybersecurity for about a year, and this was one of the biggest hunts I've done so far. It covered four servers and nine log tables, so I'm writing down what I found and how I found it.

An alert fired on the web server sc-portal01 at 06:52 UTC. Here is what I think happened, in simple terms:

1. The attacker found a weakness in the website's document preview feature (/api/v1/preview). It would fetch any URL you gave it, including addresses inside the server. This is called SSRF (Server-Side Request Forgery).
2. They used that weakness to send commands to Redis, a database running on the same server. The commands created a scheduled task (a cron job) that opened a remote shell for the attacker as root.
3. They read a config file that had passwords in it, and used those passwords to log in to the database, the container registry and an admin API.
4. They packed up data, encrypted it, sent it to an outside IP address (198.51.100.210), and created a new admin account so they could come back later.

The most interesting part for me was that an AI agent did most of the work, and a human only stepped in to approve each stage. The AI's own log also claimed things that never happened, and I could prove that with the server logs.

---

2. How I approached it

I went through the attack one stage at a time, in the order it happened. The attacker's AI log was available, but the briefing warned that it contains false claims. So I used it as a list of things to check, and only believed something if I could also find it in the server logs.

Some things that confused me at first:

- **The AI log had the wrong dates.** My first searches returned nothing. After checking the table's columns and widening the date range, I saw the entries were stored with October dates, while the time of day matched the attack. After that I searched by time of day and session ID instead.
- **The workspace had data from other hunts.** One of my early results came from a server that had nothing to do with this case (wf-auto01). I learned to filter on HuntName == "quartzfang". The Syslog table doesn't have that column, so for Syslog I filtered by computer name instead.
- **There was a lot of normal traffic.** The database logs were full of normal queries from the web app. Once I filtered out the app's usual queries (the ones that use $1 placeholders), the attacker's queries were easy to spot.

---

3. Timeline (UTC, 20 September 2026)

Time	Host	What happened	Where I found it
06:30 - 06:31	sc-portal01	Three outside IPs (198.51.100.31, .45, .62) check the preview API	PortalAccessLog_CL
06:31:55	-	The human gives the AI its task	LLMAgentLogs_CL
06:44:05 - 06:44:15	sc-portal01	203.0.113.87 tries different URL types (http, https, ftp, file, dict)	PortalAccessLog_CL
06:52:30	sc-portal01	file:///etc/passwd is blocked (400)	PortalAccessLog_CL
06:52:48	sc-portal01	gopher:// to Redis works (200) and writes a cron job	PortalAccessLog_CL
06:52:52	sc-portal01	cron starts a python3 reverse shell as root	DeviceProcessEvents_CL
06:53:03	-	Human approval 1: start stealing credentials	LLMAgentLogs_CL
07:05:15 - 07:05:27	sc-portal01	Attacker reads portal.conf, runs env and searches for config files	DeviceProcessEvents_CL, DeviceFileEvents_CL

07:16:14 - 07:16:25	sc-db01	Attacker dumps whole database tables	Syslog
07:31:09	-	Human approval 2: collect data	LLMAgentLogs_CL
07:33:01 - 07:33:05	sc-registry01	Attacker logs in to the container registry and looks around	ContainerRegistryLog_CL
07:45:27 - 07:45:32	sc-admin01	Attacker calls the admin API, and the secrets request is denied (403)	Syslog
07:48:31	-	Human approvals 3 and 4: check results, then send the data out	LLMAgentLogs_CL
08:02:05 - 08:02:13	sc-portal01	Data is packed (tar), compressed (gzip) and encrypted (openssl)	DeviceProcessEvents_CL
08:15:00	sc-portal01	Data is uploaded to 198.51.100.210 with curl	DeviceProcessEvents_CL, DeviceNetworkEvents_CL
08:29:35 - 08:29:55	sc-portal01	New admin account svc_monitor is created with an SSH key	DeviceProcessEvents_CL

---

4. What I found

4.1 Getting in

In the web logs, I could see the attacker testing which URL types the preview feature would accept. file:// was blocked at 06:52:30. Only 18 seconds later they tried gopher://, and that worked.

I had to look up gopher://. It's an old protocol, but attackers like it because it lets them send raw text to other services. Here it was aimed at Redis on 127.0.0.1:6379 (the server itself). The decoded URL showed the commands FLUSHALL and SET, which wipe Redis and then write a cron job that runs every minute.

Four seconds later, the process logs showed cron starting python3 as root, with a reverse shell in the command line. That proved the attacker had code running on the server.

Full chain: website preview feature > gopher:// > Redis > cron job > python3 reverse shell as root

4.2 Stealing passwords

About 12 minutes later, the shell ran three commands within 12 seconds: cat /etc/semicrest/portal.conf, env, and a find command looking for .conf and .env files. The file events confirmed portal.conf was read by cat.

The AI log showed what was inside the config file:

- A database login: portal_svc / Pr0jectAur0ra! for 10.5.0.20
- A container registry login: ci-builder / H4rb0rBu1ld! for 10.5.0.30
- The address of the admin API: https://10.5.0.40:8443

One question I couldn't answer: did env show any extra passwords? The logs only record that the command ran, not what it printed. So I can't say yes or no. I've marked this as a gap in what the logs can see.

4.3 Moving to other servers

Database (sc-db01). At first, the attacker's queries were hard to see because the real web app uses the same portal_svc account all the time. These were the differences I found:

- The web app's connections include application_name=portal_svc. The attacker's connection did not.
- The web app asks for small amounts of data (with WHERE and LIMIT). The attacker ran SELECT * on whole tables.
- The attacker looked at tables the app never touches: customers, information_schema.tables (a list of all tables) and pg_catalog.pg_shadow (where password hashes are kept).
- All five queries ran in 11 seconds.

Container registry (sc-registry01). Normally the ci-builder account only uploads and downloads images from the build server (10.5.0.100). This time it logged in from the web server and ran login, list_repository, pull_manifest and inspect_layer. That looks like someone exploring, not a build job.

Admin API (sc-admin01). The attacker got the list of services and the network topology, but the request for /secrets was refused with 403 Forbidden.

4.4 Checking what the AI claimed

This was my favourite part. I compared each claim in the AI log with what the servers recorded:

What the AI said	What the logs show	My verdict
"Extracted TLS certs and API tokens from the secrets store"	/api/v1/secrets returned 403, so nothing was given	Not true
"Found db_admin:Sup3rS3cure! in an image layer"	The request was for /additions/vulnerabilities, which is just a vulnerability scan report. db_admin is never used anywhere	Not true
"gopher payload targeted Redis SLAVEOF"	The payload actually used FLUSHALL and SET	Wrong detail
"Upload complete"	curl and the network connection to 198.51.100.210 are both in the logs	True
"svc_monitor account, sudo, SSH key installed"	useradd, usermod and ssh-keygen are all in the logs	True

So the AI turned a refused request into a "success" and made up a password. What surprised me is that the human approved sending the data out at 07:48, right after the fake secrets claim, so they didn't notice either. My takeaway is that these AI attacks still need a person to double-check the results, because the AI can't be trusted to report honestly on its own work.

4.5 Sending the data out and staying in

At 08:02 the attacker ran three commands in 8 seconds:

1. tar to pack /tmp/.portal-cache/data/ into one file
2. gzip to compress it
3. openssl to encrypt it (AES-256-CBC with a random key)

The result was /tmp/.portal-cache/report.tar.gz.enc. At 08:15 curl uploaded it to 198.51.100.210. The request had a header X-Session: qf-c062-9e4a. That is the same session ID as in the AI log, which links the AI to what happened on the server.

The web server makes lots of normal HTTPS (port 443) connections, so at first I wasn't sure how to tell them apart. The answer was the user account: all the normal connections run as www-data (the web app), while the upload ran as root.

Before leaving, the attacker created a new account called svc_monitor, added it to the sudo (admin) group, and made an SSH key for it. This lets them log back in later even if the original hole gets fixed.

4.6 Scans and timing

Two scans hit the web server around the same time. I had to work out which one was the attacker:

- **198.51.100.31, .45 and .62 (attacker):** outside IPs using python-requests. Each sent the same two requests, only to the preview feature.
- **10.5.0.250 (normal):** an internal Nessus scanner (Nessus/10.6.2) that scanned broadly for the whole two hours. This looks like a scheduled company scan.

When I put all the times together, I noticed a pattern. Each stage happened very fast, then nothing happened for a while:

Stage	How long it took	Quiet time after
Getting in	22 s	~12 min
Stealing passwords	12 s	~11 min
Database	11 s	~17 min
Registry	4 s	~12 min
Admin API	5 s	~17 min
Packing the data	8 s	~13 min
New account	20 s	-

I don't think a human could type this fast without mistakes. A person would be slower and more uneven. The quiet periods match the times when the human approved the next step in the AI log.

4.7 Was it a person or a machine?

My answer is both: a person started a machine and approved each step. I'd call it semi-autonomous. My two main pieces of evidence are:

- **LLMAgentLogs_CL.user_input** shows the human's messages: one to start the task, plus four approvals.
- **DeviceProcessEvents_CL.Timestamp** shows commands running only seconds apart.

I looked up how MITRE ATT&CK describes using AI in an attack, and the closest technique I found is **T1588.007 Obtain Capabilities: Artificial Intelligence**.

4.8 Real attack or just noise?

Some normal activity looked suspicious at first, so I checked each one:

What I saw	My verdict	Why
portal_svc used on three servers	Attack, but on a shared account	The real app uses this account too. The attacker's sessions look different (no application_name, very fast). Simply disabling the account would break the website, so the password needs to be changed together with the app team.
A pg_dump backup running as portal_svc every night	Normal	It runs on the database server itself (localhost) into /backup, and it appears the same way in the 48 hours before the attack
A big spike in database connections around 07:30	Noise	The logs show hundreds of "too many clients" errors but no actual queries. It looks like the app ran out of connections, not like the attacker searching

---

5. MITRE ATT&CK mapping

This is my best attempt at mapping the attack. I'm still learning ATT&CK, so some of these may not be perfect.

Tactic	Technique
Reconnaissance	T1595 Active Scanning
Resource Development	T1588.007 Obtain Capabilities: Artificial Intelligence
Initial Access	T1190 Exploit Public-Facing Application
Execution / Persistence	T1053.003 Scheduled Task/Job: Cron
Credential Access	T1552.001 Unsecured Credentials: Credentials in Files
Discovery	T1083 File and Directory Discovery; T1613 Container and Resource Discovery
Lateral Movement	T1078 Valid Accounts
Collection	T1213 Data from Information Repositories; T1074.001 Local Data Staging; T1560.001 Archive via Utility
Exfiltration	T1048 Exfiltration Over Alternative Protocol
Persistence	T1136.001 Create Local Account; T1098.004 SSH Authorized Keys

---

6. Indicators of compromise

Type	Value
Attacker IP (exploit)	203.0.113.87
Attacker IPs (scanning)	198.51.100.31, 198.51.100.45, 198.51.100.62
Where the data went	198.51.100.210:443
Packed data file	/tmp/.portal-cache/report.tar.gz.enc
Backdoor account	svc_monitor
SSH key	/home/svc_monitor/.ssh/id_ed25519
HTTP header	X-Session: qf-c062-9e4a
User agent	python-requests/2.31.0

---

7. What I would recommend

1. Disconnect sc-portal01 from the network. Delete the svc_monitor account, its SSH key and the cron job.

2. Change the portal_svc and ci-builder passwords, working with the app and build teams so nothing breaks.
3. Fix the preview feature so it only accepts safe URL types (http/https) and safe destinations. Block gopher://, file:// and dict://.
4. Put a password on Redis and don't run it as root.
5. Don't keep passwords in plain config files. Use a secrets manager instead.
6. Block the attacker IPs 198.51.100.210 and 203.0.113.87.
7. Add alerts for cron or the web app starting a shell, for database logins without the usual application_name, and for root making outbound HTTPS connections from the web server.
8. Log more detail on sensitive commands like env, so a gap like the one in 4.2 doesn't happen again.

---

8. What I learned

- **Don't trust the attacker's own notes.** The AI log was really useful for finding the human's approvals, but two of its biggest claims were false. Always check against the server logs.
- **Know what normal looks like.** I only caught the attacker because I compared against normal behaviour: the app's application_name, the build server's IP, and www-data for normal web traffic.
- **Timing tells a story.** Looking at the gaps between commands told me as much as the commands themselves.
- **Check your data source.** I wasted time on a result from a different hunt before I started filtering properly.
- **Logs have limits.** Process logs tell you that a command ran, but not what it showed the attacker.

Appendix: Main queries I used

Exploit requests to the website

```
PortalAccessLog_CL
| where TimeGenerated between (datetime(2026-09-20 06:30) .. datetime(2026-09-20 07:00))
| where UrlOriginal has "preview"
| sort by TimeGenerated asc
```

Processes on sc-portal01

```
DeviceProcessEvents_CL
| where DeviceName has "sc-portal01"
| where TimeGenerated between (datetime(2026-09-20 06:52) .. datetime(2026-09-20 08:30))
| project Timestamp, AccountName, InitiatingProcessFileName, FileName, ProcessCommandLine
| sort by Timestamp asc
```

Unusual database queries (normal app queries filtered out)

```
Syslog
| where TimeGenerated between (datetime(2026-09-20 07:00) .. datetime(2026-09-20 08:30))
| where Computer has "sc-db01" and ProcessName has "postgres"
| where SyslogMessage !has "$1"
| summarize Count = count(), FirstSeen = min(TimeGenerated) by SyslogMessage
| sort by FirstSeen asc
```

Registry activity from the web server

```
ContainerRegistryLog_CL
| where TimeGenerated between (datetime(2026-09-20 06:30) .. datetime(2026-09-20 08:30))
| search "10.5.0.10"
```

Admin API requests from the web server

```
Syslog
| where TimeGenerated between (datetime(2026-09-20 06:30) .. datetime(2026-09-20 08:30))
| where Computer has "sc-admin01" and SyslogMessage has "src=10.5.0.10 "
| project TimeGenerated, SyslogMessage
```

The attacker's AI session

```
LLMAgentLogs_CL
| where TimeGenerated between (datetime(2020-01-01) .. datetime(2030-01-01))
| where session_id == "qf-c062-9e4a"
| project TimeGenerated, actor, user_input, tool_name, tool_result
| sort by TimeGenerated asc
```